

Stage 1 — Building the Data Layer

Technical Report on Python Architecture, Implementation & Benchmarking Analysis

Course:	Big Data — B.Sc. Data Science and Engineering (ULPGC)	Date:	October 2026
Team:	TheScratchers	Branch:	ft/python-setup (Python 3.9+)
Author:	Aimar Daniel Alejandro Santana	Repository:	github.com/TheScratchers/stage_1

1. Executive Summary & Specification Compliance

This report presents the architecture, implementation details, and empirical benchmark results of the **Data Layer (Stage 1)** developed in Python for the Project Gutenberg search engine. The implementation follows the cross-language benchmark contract (*shared/CONTRACT.md*) agreed with the Java and C++ teammates, so that the numbers reported here are directly comparable with theirs: the same 20 real Gutenberg books, the same tokenizer rule, the same 10-word query workload, and the same synthetic scales.

Requirement (Contract)	Python Component	Implementation & Coverage
Shared 20-book dataset	<code>download_shared_dataset.py</code> , <code>shared_contract.py</code>	Downloads exactly the 20 contract books (<code>./shared/books.txt</code>) into <code>datalake_shared/</code> ; idempotent, re-run-safe. All three benchmarks replicate this real content across synthetic ids rather than inventing their own corpus.
Ingestion & Header/Body Split	<code>download_book.py</code>	Fetches a book from Project Gutenberg and splits it into <code><id>.header.txt</code> / <code><id>.body.txt</code> , consumed by every datalake layout and by <code>metadata.py</code> .
3 Datalake Layouts	<code>datalake.py</code> , <code>datalake_book.py</code> , <code>datalake_batch.py</code>	Time-based (<code>YYYYMMDD/HH/</code>), book-based (<code><id>/</code>), and batch-based (<code>batch_N-M/</code> , 1,000-id ranges) — three interchangeable path-resolution strategies sharing the same ingestion/benchmark harness.
Tokenizer / normalization rule	<code>inverted_index*.py</code> (<code>TOKEN_PATTERN</code>)	Lowercases first, then extracts maximal runs of <code>[A-Za-z]+</code> ; no stemming, no stopwords removal — exactly the rule mandated in <i>CONTRACT.md</i> Section 2.
3 Inverted Indexes	<code>inverted_index.py</code> , <code>inverted_index_hierarchical.py</code> , <code>inverted_index_sqlite.py</code>	Monolithic JSON file, hierarchical per-letter/per-term files, and a SQLite table with a composite (<code>term</code> , <code>book_id</code>) primary key — the team's three agreed structures. The third structure was originally MongoDB; mid-stage the team (Python, Java, C++) jointly replaced it with SQLite, matching the Java implementation's existing schema, so the structure is identical across all three languages rather than three independent choices.

Standard Query Workload	<code>shared_contract.py</code> (<code>load_words()</code>)	Parses the 10 fixed terms from <code>../shared/words.txt</code> (4 common / 4 medium / 2 rare), used identically for query and update benchmarks.
Metadata Datamart (SQLite)	<code>metadata.py</code>	Schema with <code>book_id</code> primary key, <code>title/author/language</code> , plus <code>body/header</code> paths; <code>upsert_book</code> , <code>find_by_id</code> , <code>find_by_author</code> exercised directly (no idealized bulk-insert bypass).
Metadata Benchmark (contract extension)	<code>benchmark_metadata.py</code>	Insertion speed, <code>find_by_id</code> , and <code>find_by_author</code> (Dickens/Carroll, timed separately) at the 4 contract scales (100 / 1,000 / 10,000 / 100,000).
Control Layer & State Tracking	<code>control.py</code>	Orchestrates one download or one indexing step at a time, tracking progress in <code>control/downloaded_books.txt</code> and <code>indexed_books.txt</code> to avoid duplicate work.
Datalake / Index / Metadata reporting (Section 6)	<code>benchmark_datalake.py</code> , <code>benchmark_inverted_index.py</code> , <code>benchmark_metadata.py</code>	Every metric Section 6 lists — write throughput, lookup cost, incremental-processing cost, recovery behavior and storage overhead for the datalake; indexing speed, query performance, update performance, memory/disk usage for the index — is reported at every one of the 3 scales , not as a single flat snapshot.

This last row is deliberate: every metric in Sections 4.1–4.3 below is reported at the full 3-scale progression (100 / 1,000 / 10,000 books, plus the 100,000-book metadata tier), not as a single flat snapshot. A note on the third inverted-index structure specifically: it was originally MongoDB, and an earlier version of this report benchmarked it as a live database. Mid-stage, the team re-evaluated that choice — MongoDB proved heavy and unreliable on one member's machine, and the comparison across languages only makes sense if all three use the *same* third structure — so Python, Java and C++ jointly replaced it with SQLite (matching the schema already in use on the Java side). Section 4.2 below reflects that decision.

2. System Architecture & Design Principles

The codebase separates raw storage (**Datalake**), query-optimized derived stores (**Datamarts**), and pipeline orchestration (**Control Layer**), with a thin **shared-contract** layer gluing the benchmark scripts to the cross-language agreement.

- **Datalake Layer:** the repository for ingested book texts. Exposes three interchangeable partitioning layouts (time-based, book-based, batch-based), each a small module providing a single `*_path()` function; all three are driven by the same `download_book()` call and evaluated by the same benchmark harness, so the only thing that differs between them is where a book's two files end up on disk.
- **Metadata Datamart:** extracts title, author and language from each book's header via regular expressions and stores them in SQLite, keyed by `book_id`, alongside pointers back to the body/header files in the datalake.
- **Inverted Index Datamart:** tokenizes body texts under the shared contract's rule and maintains postings (term → `book_ids`) across three physical representations with very different cost profiles: a single JSON file (cheap once loaded, expensive to touch at scale), a folder-per-letter/file-per-term layout (no load step, but one filesystem call per lookup), and a SQLite table with a composite (`term`, `book_id`) primary key (an indexed, query-engine-backed lookup, with no external database process to run — unlike the MongoDB collection this structure replaced).

- **Control Layer:** a minimal orchestrator that advances the pipeline one step at a time (download a new book, or index one already-downloaded one — never both in the same step), persisting progress in two plain-text files so a run can be stopped and resumed without re-processing anything.
- **Shared-contract layer:** `shared_contract.py` is the single place that parses `../shared/CONTRACT.md`'s companion files (`books.txt`, `words.txt`) and the two fixed metadata-benchmark authors, so the exact same 20 books and 10 query words are used identically by all three benchmark scripts instead of being copy-pasted three times.

3. Modular Codebase Breakdown

Each module has one responsibility: one file per datalake layout, one file per index structure, implementation code kept separate from its benchmark harness. Total: **2,283 LOC** across 14 source files, plus a **954-LOC** pytest suite (9 files, 70 tests) covering every module, including the SQLite-backed inverted index — exercised directly via Python's standard library, no external database needed (unlike the MongoDB structure it replaced, which could only be exercised manually against a live instance).

File	LOC	Responsibility
<code>shared_contract.py</code>	47	Parses the shared contract's <code>books.txt/words.txt</code> and the 2 fixed metadata-query authors — the single source of truth all 3 benchmark scripts read from.
<code>datalake_book.py</code>	55	Book-based datalake layout: one dedicated directory per book.
<code>datalake_batch.py</code>	71	Batch-based datalake layout: books grouped into fixed-width id-range folders (default 1,000 ids/folder).
<code>datalake.py</code>	73	Time-based datalake layout (YYYYMMDD/HH/) plus the shared <code>ingest_book()</code> entry point.
<code>download_book.py</code>	104	Fetches one book from Project Gutenberg and splits it into header/body text files.
<code>download_shared_dataset.py</code>	105	Idempotently downloads the 20 contract books into <code>datalake_shared/</code> .
<code>inverted_index.py</code>	128	Monolithic single-JSON-file inverted index: build, search, incremental update.
<code>inverted_index_hierarchical.py</code>	141	Folder-per-letter/file-per-term inverted index, with fine-grained per-term update.
<code>inverted_index_sqlite.py</code>	162	SQLite-backed inverted index: a single <code>(term, book_id)</code> composite-primary-key table, mirroring the Java implementation's schema; INSERT OR IGNORE-based update.
<code>metadata.py</code>	174	SQLite metadata datamart: schema, header parsing, upsert/find_by_id/find_by_author.
<code>control.py</code>	188	Pipeline orchestrator; tracks downloaded/indexed state across two control files, incrementally updating all three inverted-index structures (JSON, hierarchical, SQLite) on every indexing step.
<code>benchmark_metadata.py</code>	213	Metadata benchmark harness: insertion speed, query performance, scalability at 4 scales.
<code>benchmark_datalake.py</code>	328	Datalake benchmark harness: write throughput, lookup, incremental processing, recovery, storage overhead, 3 structures × 3 scales.

benchmark_inverted_index.py	494	Inverted-index benchmark harness: build time, 10-word query workload, update performance, peak memory, storage overhead, 3 structures × 3 scales — a single run now produces all 3 structures' results.
-----------------------------	-----	---

4. Experimental Results & Benchmark Analysis

All benchmarks were executed on the author's Windows 11 machine, against the 20 real Project Gutenberg contract books (shared/books.txt, fetched via download_shared_dataset.py), replicated across synthetic book_ids at the 3 contract scales (100 / 1,000 / 10,000 books). Every run writes its disposable synthetic files to a scratch folder outside the project tree, so results reflect genuine I/O cost rather than cloud-sync interference.

4.1 Datalake Layout Benchmark — full scale progression

Unlike a single-point snapshot, every metric below is reported at all 3 contract scales for all 3 layouts, so the actual scalability trend — not just one data point — is visible.

Layout	Scale	Write (books/s)	Lookup avg (ms)	Dirs created	Max files/dir
time_based (YYYYMMDD/HH/)	100	207.0	14.35	2	100
book_based (<id>/)	100	225.3	13.44	100	1
batch_based (batch_N/)	100	190.7	13.13	1	100
time_based	1,000	225.3	12.86	2	1,000
book_based	1,000	199.4	13.57	1,000	1
batch_based	1,000	208.4	13.44	1	1,000
time_based	10,000	201.8	15.23	2	10,000
book_based	10,000	201.4	13.41	10,000	1
batch_based	10,000	233.6	12.54	10	1,000

Layout	Scale	Incr. detect (s)	Incr. write (s)	Recovery resumed	Recovery (s)	Recovery OK
time_based	100	0.0028	0.0342	50	0.233	✓
book_based	100	0.0020	0.0381	50	0.260	✓
batch_based	100	0.0019	0.0343	50	0.198	✓
time_based	1,000	0.0266	0.454	500	2.426	✓
book_based	1,000	0.0282	0.455	500	2.427	✓
batch_based	1,000	0.0298	0.568	500	2.528	✓
time_based	10,000	0.2756	4.199	5,000	22.141	✓
book_based	10,000	0.2797	4.861	5,000	24.782	✓
batch_based	10,000	0.1748	3.740	5,000	22.421	✓

Analysis:

- **Write throughput is close at small scale, diverges at 10,000:** at 100–1,000 books the three layouts stay within ±10–15% of each other (dominated by per-file I/O rather than directory structure), but at 10,000 books batch_based pulls ahead (233.6 books/s vs. ~201.4–201.8 for the other two) — it caps every directory at 1,000 entries (growing the directory count instead: 1 → 1 → 10 as scale increases tenfold each time), avoiding

both `time_based`'s single hot directory (2 dirs holding all 10,000 files at the top scale) and `book_based`'s one-directory-per-book overhead (10,000 directory creations).

- **Storage/structural overhead is the direct trade-off the contract asks for:** `time_based` keeps directory count flat (2) regardless of volume, at the cost of an unbounded single directory; `book_based` keeps every directory at exactly 1 file, at the cost of one directory per book (10,000 at the top scale); `batch_based` trades a configurable middle ground (1,000 files/directory by default).
- **Recovery cost scales with how many existing entries must be re-verified, not just with total volume:** all three layouts correctly resumed every already-ingested book without re-writing it (`recovery_ok = true` at every scale), but `book_based` was consistently the slowest to recover at 10,000 books (24.78 s vs. 22.14–22.42 s for the other two), since resuming means statting 10,000 individual per-book directories instead of scanning a shared one.
- **Incremental-processing cost (detecting + writing new books without re-reading the existing set) stays a small fraction of a full rebuild at every scale** — at 10,000 books, detecting and writing ~1,000 new candidates takes 0.17–0.28 s + 3.7–4.9 s, versus 42.8–49.6 s for a full 10,000-book write — confirming all three layouts support cheap append-only ingestion on top of an already-populated datalake.

4.2 Inverted Index Benchmark — all 3 structures (JSON, Hierarchical, SQLite)

The vocabulary size (**35,008 unique terms**) is identical across every scale and every structure: synthetic volume is generated by replicating the 20 real books' content across new `book_ids` (one synthetic book per position i , cycling through the 20 real books at position $i \bmod 20$ — see *shared/CONTRACT.md* Section 4.1), so the set of distinct words never grows — only each term's postings list does. That is exactly what should drive the cost differences below. These numbers supersede an earlier version of this report, which benchmarked MongoDB as the third structure before the team's mid-stage decision to replace it with SQLite across all three languages (see Section 1).

Structure	Scale	Build (s)	Peak memory (MB)	Disk footprint (MB)	Avg lookup (ms)	Update cost (s)
json_monolithic	100	19.87	67.97	5.81	0.0029	0.490
hierarchical	100	43.52	67.96	5.29	1.881	6.857
sqlite	100	17.43	96.76	28.03	1.014	0.222
json_monolithic	1,000	184.19	413.50	53.84	0.0017	4.006
hierarchical	1,000	173.33	375.88	53.32	8.338	15.890
sqlite	1,000	166.81	790.54	297.03	1.695	0.332
json_monolithic	10,000	2021.93	4626.81	534.09	0.0048	47.975
hierarchical	10,000	1487.34	4109.00	533.57	11.943	28.443
sqlite	10,000	1582.20	8384.00	2955.63	34.531	1.779

In-Depth Analysis & Trade-offs:

- **SQLite is the cheapest structure to update at every single scale, by a wide and growing margin** (0.22 s → 0.33 s → 1.78 s) — roughly 16× faster than hierarchical (6.86 s → 15.89 s → 28.44 s) and 27× faster than JSON (0.49 s → 4.01 s → 47.98 s) at 10,000 books. A single batched INSERT OR IGNORE transaction for just the new book's terms is far cheaper than hierarchical's per-term read-modify-write of existing files, or JSON's full-file rewrite. This is the clearest practical argument for SQLite as the structure wired into `control.py`'s live, incremental pipeline.

- **SQLite's lookup cost crosses over from fastest to slowest as scale grows:** at 100 and 1,000 books it beats hierarchical (1.01 ms vs. 1.88 ms, and 1.70 ms vs. 8.34 ms), but at 10,000 books it becomes the *most* expensive per query of the three (34.53 ms, vs. 11.94 ms for hierarchical) — a genuine, non-monotonic finding, not a measurement artifact. The most likely explanation is disk-page-cache pressure: the SQLite file itself has grown to ~2.9 GB at this scale (see disk footprint below), large enough that random-access B-tree lookups start paying real disk I/O, whereas hierarchical's lookup touches one small, cheap-to-open file regardless of how large the index as a whole has grown.
- **SQLite's disk footprint is 5–6× larger than JSON or hierarchical at every scale** (28.0 MB vs. ~5.3–5.8 MB at 100 books; 2,955.6 MB vs. ~534 MB at 10,000 books), because this structure stores one full row per (term, book_id) posting — with SQLite's own B-tree/page overhead per row — instead of a single compact list of ids per term the way JSON and hierarchical do. This is an inherent, expected trade-off of a row-store schema built for queryability, not an implementation inefficiency.
- **SQLite's peak memory is consistently higher than JSON or hierarchical at every scale** (96.8 MB vs. ~68 MB at 100 books; 8,384.0 MB vs. ~4,109–4,627 MB at 10,000 books, close to double), which is specifically a consequence of this implementation's `build_index()`: after scanning every book into the usual `term → book_ids` dictionary, it additionally flattens that whole structure into a second, parallel list of (term, book_id) tuples before handing it to `executemany()` — so both representations briefly coexist in memory at the moment `tracemalloc` captures its peak. JSON and hierarchical never build this second copy.
- **Build time no longer separates the structures as cleanly as it did with MongoDB.** SQLite is fastest at 100 and 1,000 books (17.43 s, 166.81 s) but middling at 10,000 (1,582.20 s, between hierarchical's 1,487.34 s and JSON's 2,021.93 s). SQLite's build time scales almost perfectly linearly with volume (9.6× and 9.5× for each 10× scale step, tracking total posting-row count), while hierarchical's scales sub-linearly at first (its build always writes the same 35,008 term files per run, so only each file's growing content — not the number of file-open calls — adds cost as volume increases), which is also why hierarchical overtakes both other structures to become the fastest build at 10,000 books despite being the slowest at 100.
- **JSON monolithic's query cost stays negligible (<0.005 ms even at 10,000 books)** once the one-time cold load has happened (0.137 s → 0.89 s → 10.56 s, growing with file size), since every lookup after that is a plain in-memory dict access — but its update cost still grows almost linearly with the full file size, because `update_index()` for this structure re-serializes and rewrites the *entire* file on every single change, making it the least suitable of the three for frequent small updates at scale.

4.3 Metadata Storage Benchmark (SQLite stress test, contract Section 5)

Scale (books)	Insert (s)	Throughput (books/s)	find_by_id (ms)	Dickens lookup (ms)	Carroll lookup (ms)	DB size (KB)
100	0.694	144.1	0.525	0.505	0.507	24
1,000	4.969	201.2	0.399	0.827	0.818	132
10,000	51.119	195.6	0.650	8.00	5.28	1,224
100,000	525.891	190.2	0.572	57.38	34.64	12,164

Analysis:

- **Insertion throughput stays essentially flat (144–201 books/s) across 3 orders of magnitude:** `upsert_book()` opens, commits and closes its own SQLite connection on every single call — the actual production code path used by `control.py`, not an idealized bulk-insert bypass — so per-call connection overhead dominates at every scale and total insert time scales almost perfectly linearly with row count (0.69 s → 525.9 s, a ~758x increase for a 1,000x row-count increase).

- **find_by_id stays flat and sub-millisecond at every scale** (0.40–0.65 ms across a 1,000x table-size range), confirming SQLite's primary-key B-tree index makes this lookup's cost independent of table size.
- **find_by_author scales sharply with table size for both fixed authors**, since this query has no supporting index (a LIKE substring match performs a full table scan): the repeated author (*Charles Dickens*, 3 of the 20 real books, ~15% of synthetic rows) grows from 0.51 ms at 100 rows to 57.38 ms at 100,000 rows (~113x), while the unique author (*Lewis Carroll*, 1 of 20, ~5% of rows) grows from 0.51 ms to 34.64 ms (~68x) over the same range — confirming the contract's intended effect: at scale, the more frequent author's query costs noticeably more than the rarer one's (57.38 ms vs. 34.64 ms at 100,000 rows), a gap that barely exists yet at 100 rows.
- **Storage overhead per row shrinks as the table grows** (240 B/row at 100 books down to ~124 B/row at 100,000 books), reflecting SQLite's fixed page-level overhead being amortized across more rows.

5. Execution & Verification Guide

The Python implementation requires Python 3.9+ and the dependencies in `requirements.txt` (requests only — SQLite is part of Python's standard library, so no external database or container is needed for any of the three inverted-index structures). All commands assume the current directory is `python/`.

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. One-time setup: download the 20 shared contract books
python src/download_shared_dataset.py

# 3. Single-book ingestion / indexing (manual smoke test)
python src/datalake.py 1342 5 11
python src/metadata.py
python src/inverted_index.py
python src/inverted_index_hierarchical.py
python src/inverted_index_sqlite.py
python src/control.py 3

# 4. Automated benchmarks (contract scales, default if omitted)
python src/benchmark_datalake.py 100,1000,10000
python src/benchmark_inverted_index.py 100,1000,10000
python src/benchmark_metadata.py 100,1000,10000,100000

# 5. Test suite (70 tests, no network/external database required)
pip install pytest
python -m pytest tests/ -v
```

A single invocation of `benchmark_inverted_index.py` now produces all 3 structures' results in one run — unlike the earlier MongoDB-based version, which needed a second, separate `--mongo` run against a live database instance.

Results are written as JSON to `datamarts/benchmark_datalake_results.json`, `datamarts/benchmark_inverted_index_results.json` and `datamarts/benchmark_metadata_results.json` — the three files committed to this branch and the exact source data for every table in Section 4 above.